LLMOps Core

Observability, Evaluation & Security

Ollama + OpenAI / retain the RAGOps foundation
19 slides / tools, failure drills and evidence

Four operating questions / one evidence pack

Observe / Datadog; Phoenix alternative

Where did latency, token usage or the chain failure originate?

Evaluate / Ragas; DeepEval alternative

Are claims supported, required contexts present and regressions gated?

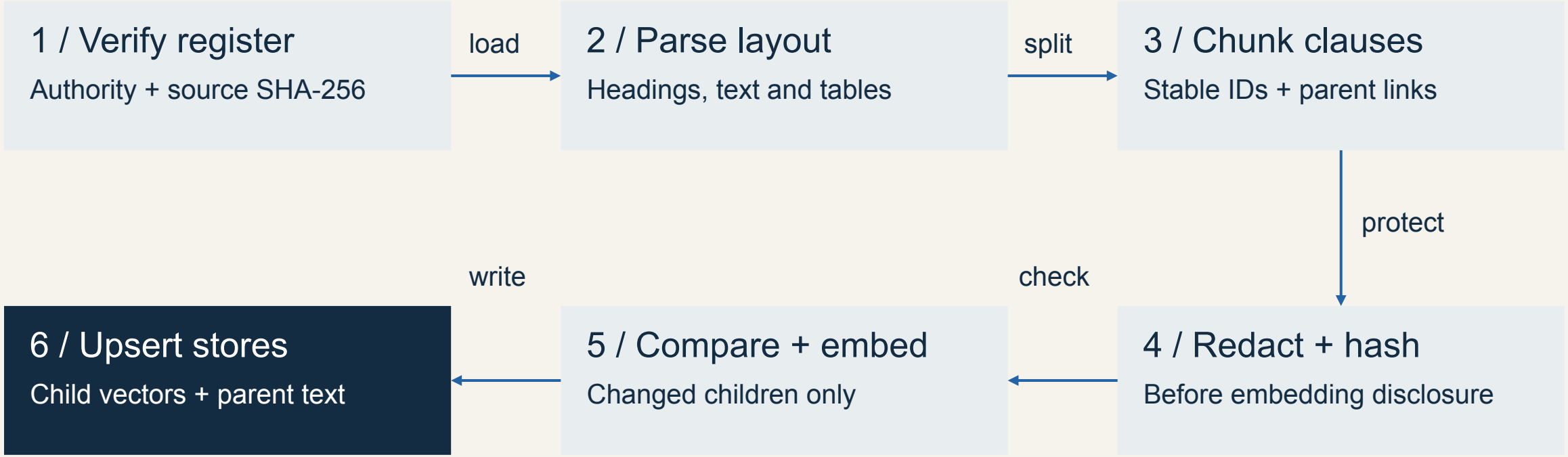
Route / LiteLLM; Portkey alternative

Which backend ran, when did fallback occur and was the cache valid?

Protect / Guardrails AI; NeMo alternative

Were PII, injection attempts and invalid outputs handled correctly?

RAGOps recap / publish verifiable evidence



Capture per-document outcome and changed counts; redaction precedes provider disclosure.

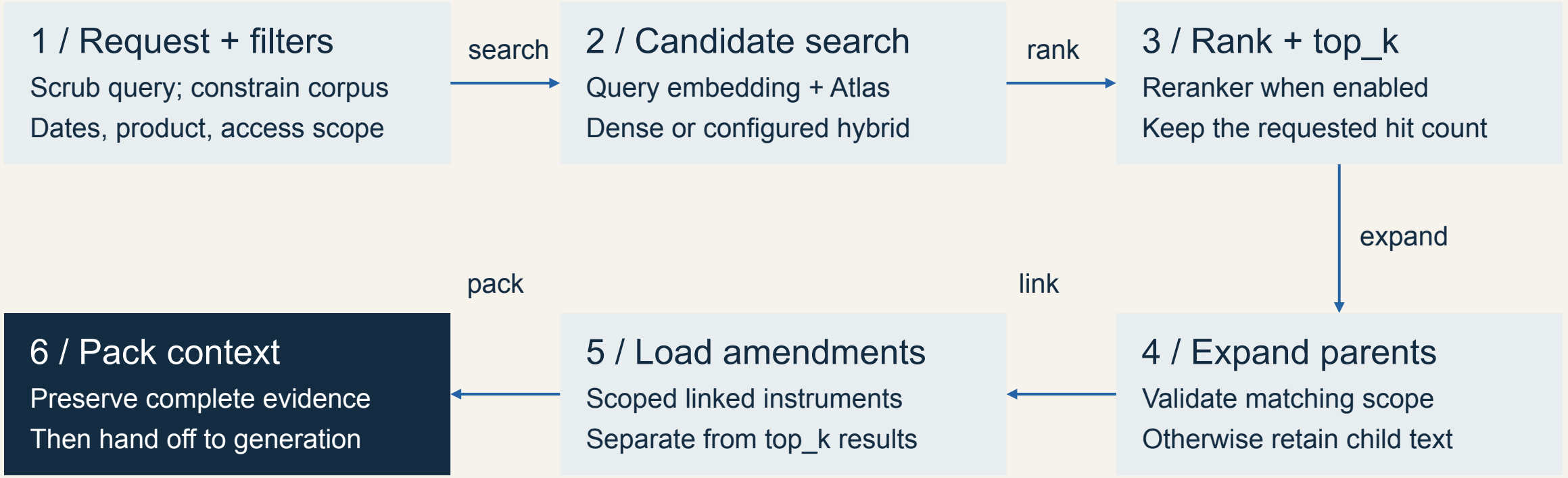
Practical / chunking and repeat ingestion

```
python -m training.session2.chunks
python -m training.session2.ingest_twin

chunk candidates 300 / 1200 / 5000 → 40 / 32 / 32
embedded counts across four runs → 32 / 0 / 32 / 0
```

Submit: preserved table, child/checksum/parent IDs and a version-change explanation.

Retrieve scoped and complete evidence



Numbered path: baseline retrieval. Hybrid search and reranking are explicit configuration branches.

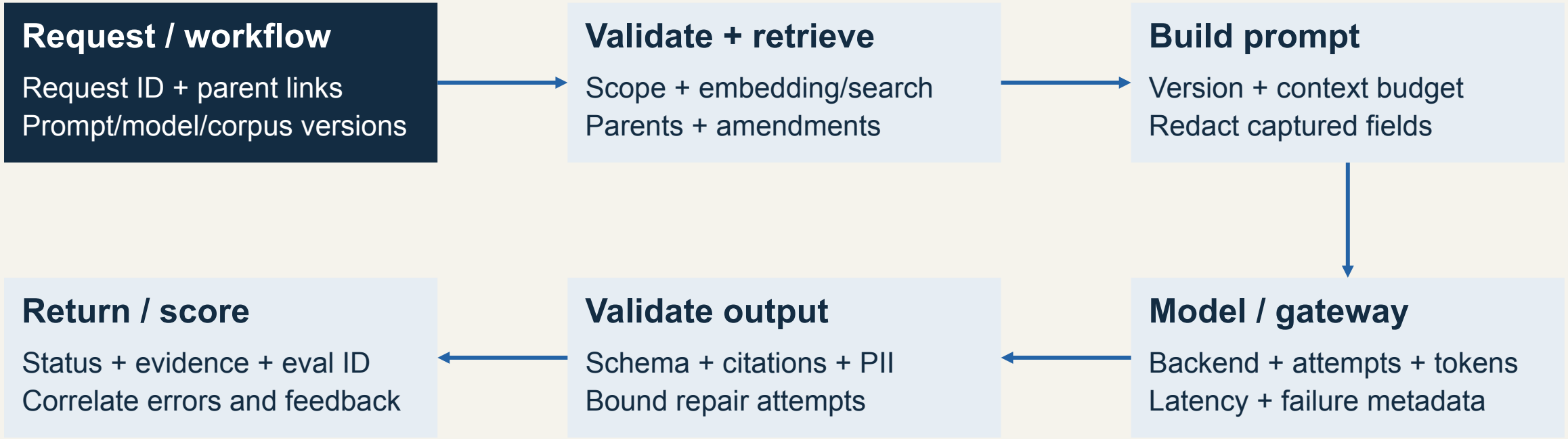
Practical / keep the policy regression baseline

```
python -m training.showcase --session 2
python -m training.session2.recall

June 0.35% → July 0.30%; wrong product abstains
one of two references → recall 0.5; rank 4 → RR 0.25
```

Retain dates, product, complete conditions and citation IDs as regression evidence.

Observability / follow one request across stages



Reference trace map: arrows show request order; inspect which spans actually exist.

Measure latency, usage and failure signals

Latency

Stage duration + request p50/p95/p99; TTFT/TPOT need token timestamps.

Usage and cost

Input/output/cached tokens, actual model/backend and measured unit cost.

Failure taxonomy

Timeout / rate limit / retrieval miss / schema error / invalid citation.

Capture policy

Redact before export; define sampling, retention and review ownership.

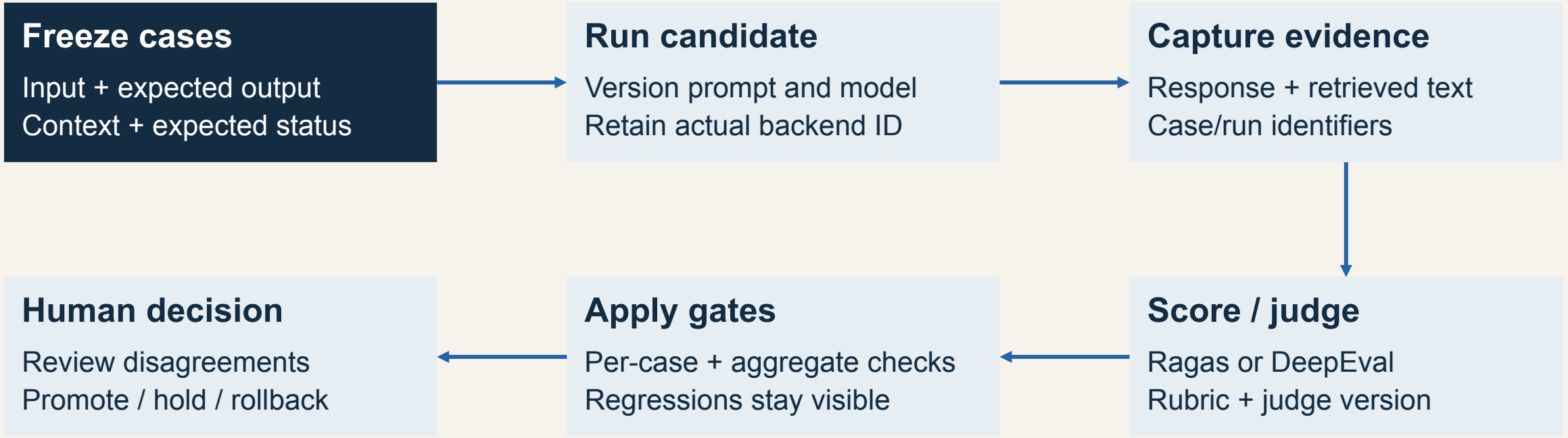
Practical / inspect a correlated Datadog trace

```
uv run --no-sync pytest -q tests/test_telemetry.py

Offline: captured spans + fake HTTP + scoped prompt metadata
Live: ddtrace-run + existing Datadog Agent configuration
Locate root → child spans; explain the slow or failing stage
```

Submit: trace/parent identity, durations, token fields, failure and capture-policy gaps.

EvalOps / build the evaluation harness



Evaluation flow: keep the dataset, reference, judge and experimental scope comparable.

Evals / separate coverage from supported claims

Faithfulness

Are the answer claims supported by the supplied retrieved context?

Context recall

Does retrieved evidence cover the claims required by the reference?

Correctness / relevance

Is the answer right for the task, and does it answer the question?

Calibration and release

Compare human labels; inspect bias, disagreement and failing cases.

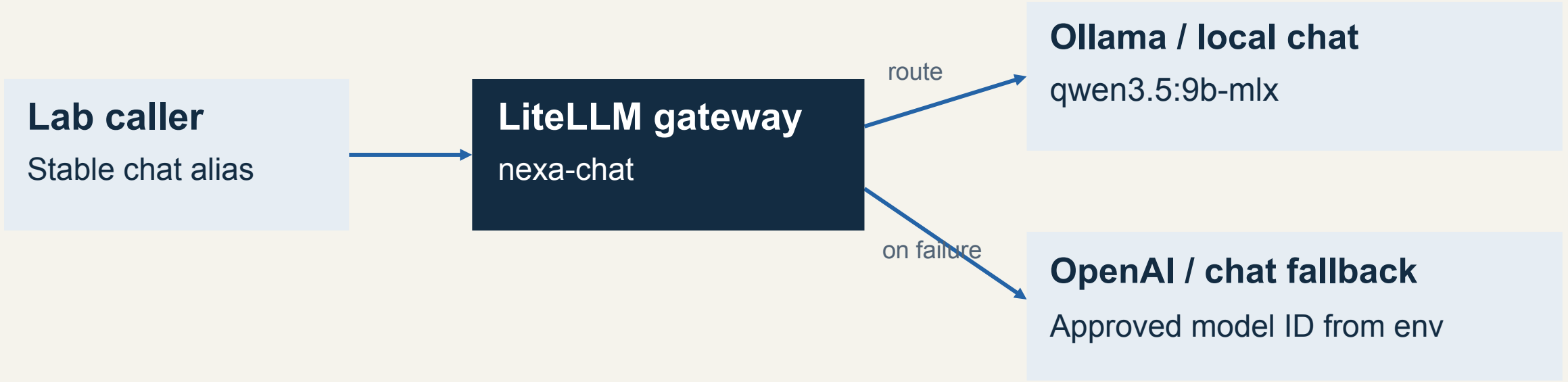
Practical / score candidates and enforce gates

```
python -m training.session3.gates

Fixture: healthy PASS; retrieval / generation faults FAIL
Live Ragas: compare Qwen chat with the OpenAI chat alias
Freeze contexts + reference + judge + metric embeddings
```

Submit: per-case scores, judge/rubric identity, failure class and HOLD/PASS rationale.

Gateway / Ollama and OpenAI behind stable aliases



Embeddings: qwen3-embedding:0.6b | OpenAI text-embedding-3-small

Chat can fail over; embedding families need separate compatible corpora and indexes.

LiteLLM / explicit routing and bounded failure

```
local chat: ollama_chat/qwen3.5:9b-mlx
local embed: openai/qwen3-embedding:0.6b [local /v1]
router_settings:
num_retries: 1
timeout: 30
fallbacks: [{nexa-chat: [nexa-chat-openai]}]
routing_strategy: simple-shuffle
```

Full four-alias YAML in training/session2/litellm.yaml; OpenAI ID/key come from env.

Practical / fail over, balance and cache

Fail over

In a temp config, break one chat URL; record attempts and actual fallback.

Balance

Use two healthy deployments under one alias; count uncached requests.

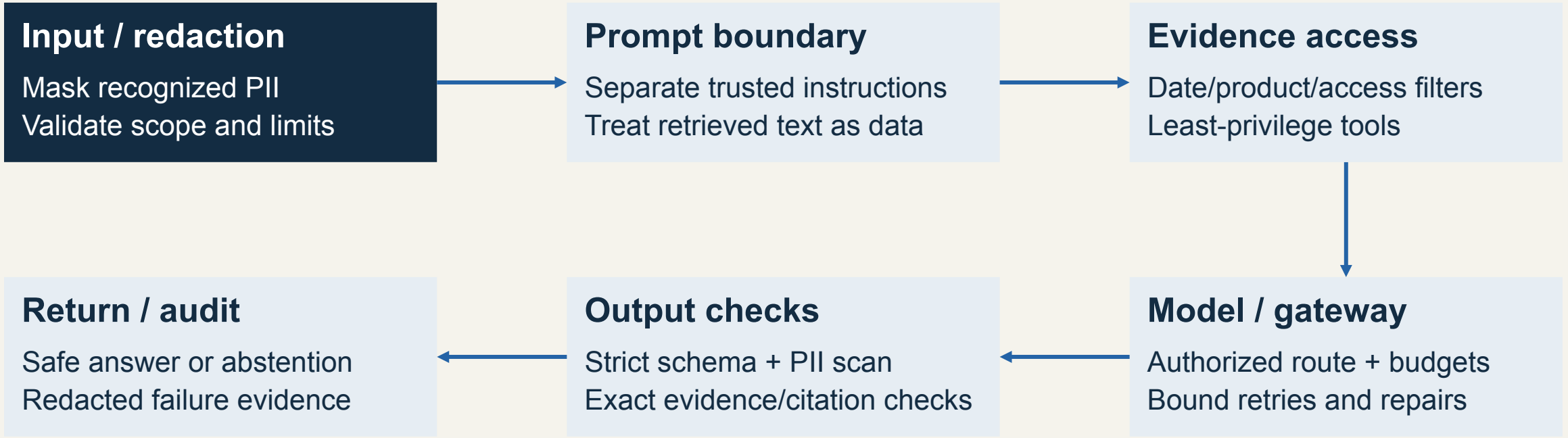
Cache

Repeat identical synthetic input; confirm a hit from upstream-call evidence.

Invalidate

Change prompt/corpus/scope version; require a fresh result or disable caching.

Guardrails / validate at each trust boundary



Defense in depth: each check has a distinct purpose; a schema alone cannot prove truth.

Practical / PII and prompt injection defenses

```
python -m training.session2.redact
python -m training.session3.prompt_boundaries

Recognized PII → typed placeholders before disclosure
Injected XML tags remain data; trusted system role retained
```

Submit: benign/adversarial cases, expected vs observed behavior and coverage gaps.

Practical / strict output with Guardrails AI

```
from guardrails import Guard
from app.models.answer import AnswerDraft
guard = Guard.for_pydantic(output_class=AnswerDraft)
outcome = guard.parse(llm_output=raw_json, num_reasks=0)
assert outcome.validation_passed

python -m training.session3.answers # offline app rails
```

Reject bad status/schema; separately reject output PII and invented source quotes.

Session 2 / operating-readiness exit gate

RAGOps + trace pack

Versioned evidence, policy regressions, parent IDs and observed stage signals.

Evaluation pack

Fixed cases, metric/judge IDs, failures and a defended release decision.

Routing + security pack

Actual backends, fallback/cache matrix, negative cases and remaining gaps.